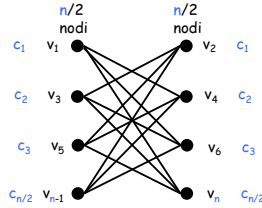


Esercizio 1

Si consideri il grafo bipartito in figura:



Sappiamo che ogni grafo bipartito è 2-colorabile; ma seguendo l'algoritmo sui nodi nell'ordine $v_1, v_2, \dots, v_n$, vengono assegnati $n/2$ colori come descritto in figura, e quindi tale algoritmo non è ottimo.

Esercizio 2

Sia $D[i, j]$ il valore che posso ottenere dai primi i elementi, avendo la disponibilità residua di al più j elementi consecutivi. In altre parole, per via di selezioni precedenti, posso continuare a scegliere fino a j elementi, ma poi dovrò saltare un elemento. La soluzione del problema originale si trova in $D[n, k]$.

Una formulazione ricorsiva è la seguente:

$$D[i, j] = \begin{cases} 0 & i = 0 \\ D[i-1, k] & i > 0 \wedge j = 0 \\ \max\{D[i-1, k], D[i-1, j-1] + V[i]\} & i > 0 \wedge j > 0 \end{cases}$$

In pratica, se $j = 0$ si è costretti a saltare, mentre se $j > 0$ si può scegliere di saltare o non saltare un elemento; se si decide di saltare, il numero di elementi consecutivi selezionabili si resetta a k ; se si decide di non saltare, restano a disposizione $j - 1$ elementi consecutivi e bisogna sommare il valore selezionato.

E' possibile risolvere il problema con memoization nel modo seguente:

kOccurrence(integer[] V, integer i, integer j, integer[][] D)
<pre> if $i \leq 0$ then return 0 if $D[i, j] = \perp$ then if $j = 0$ then $D[i, j] \leftarrow \text{kOccurrence}(V, i-1, k, D)$ else $D[i, j] \leftarrow \max(\text{kOccurrence}(V, i-1, k, D), \text{kOccurrence}(V, i-1, j-1, D) + V[i])$ return $D[i, j]$ </pre>

La complessità è pari a $O(nk)$, in quanto è necessario riempire tutta la tabella.

Esercizio 3

(1) Per calcolare il numero di disposizioni $D(n)$, è semplice definire una relazione di ricorrenza definita sulla dimensione n . Ci sono due possibilità: viene collocata una tessera verticale, e si calcolano le disposizioni per $n - 1$; oppure due tessere orizzontali, e si calcolano le disposizioni per $n - 2$. I risultati di tali calcoli vanno sommati. Per $n = 1$ e $n = 0$, il numero di disposizioni è pari a 1.

$$D(n) = \begin{cases} 1 & n \leq 1 \\ T(n-1) + T(n-2) & n > 1 \end{cases}$$

Ma questa non è altro che la sequenza di Fibonacci, spostata di un'unità:

countDomino(integer n)
return $\text{Fibonacci}(n+1)$

Il costo è quindi $O(n)$.

(2) Per stampare le disposizioni, è necessario utilizzare la tecnica del backtrack. Al solo fine di semplificare il codice ed evitare complesse funzioni di stampa, utilizziamo come vettore S delle scelte una matrice $2 \times n$.

```
printDomino(integer[][] S, integer i)
  if i = 0 then
    print S                                     % Stampa la matrice  $2 \times n$ 
  if i ≥ 1 then
    S[1][i] ← S[2][i] ← i                     % Dispone la tessera  $i$ -esima verticalmente
    printDomino(S, i - 1)
  if i ≥ 2 then
    S[1][i] ← S[1][i - 1] ← i                 % Dispone le tessere  $i$  e  $i - 1$  orizzontalmente
    S[2][i] ← S[2][i - 1] ← i - 1
    printDomino(S, i - 2)
```

La chiamata iniziale è `printDomino(S, n)`.

Poichè si dovranno contare $F(n + 1)$ disposizioni, tale numero è limitato superiormente da $O(2^n)$ e la complessità è pari a $O(n \cdot 2^n)$, per via del costo della stampa.

Esercizio 4

L'algoritmo che proponiamo agisce solo se la capacità dell'arco (u, v) è stata saturata dal flusso che passa attraverso di esso; in caso contrario, non c'è nulla da fare: il flusso massimo definito sulla capacità c è anche il flusso massimo sulla capacità modificata.

Se la capacità è stata saturata, la riduzione di un'unità di capacità richiederà una corrispondente riduzione di un'unità di flusso su (u, v) . Ma per la conservazione del flusso, sarà necessario ridurre parimenti il flusso in uno degli archi (v, v') uscenti da v , e in uno degli archi (u', u) entranti in u . Ma questo comporterà un effetto a cascata sugli archi entranti in u' e gli archi uscenti da v' , che si interromperà solo nella sorgente e nel pozzo (che non sono soggetti alla conservazione del flusso). E' necessario quindi individuare un cammino dalla sorgente a u , e un cammino da v al pozzo, definiti sugli archi tali per cui $f[u, v] > 0$ e diminuire di un'unità tutti gli archi lungo questi due cammini. Questo è realizzato dalla funzione ricorsiva `reduceFlow(F, n, x, y)`, che riduce il flusso di 1 da x a y .

Il flusso così ottenuto è un flusso che rispetta la nuova matrice di capacità; ma non è detto che sia massimo. Occorre quindi lanciare una

nuova ricerca del cammino aumentante a partire dalla sorgente, per scoprire eventuali altre strade.

```
reduceFlow(integer[][] c, integer n, integer s, integer p, integer[][] f, integer u, integer v)
```

```

c[u, v] ← c[u, v] − 1
if c[u, v] < f[u, v] and f[u, v] > 0 then
    f[u, v] ← f[u, v] − 1
    f[v, u] ← f[v, u] + 1
    reduceDFS(F, n, s, u)
    reduceDFS(F, n, v, p)
    r = c − f
    g ← cammino-aumentante(r, n, s, p)
    f ← f + g

```

```
reduceDFS(integer[][] F, integer n, integer x, integer y)
```

```

if x = y then return true

for w ← 1 to n do
    if F[x, w] > 0 then
        if reduceDFS(F, n, w, y) = true then
            F[x, w] ← F[x, w] − 1
            F[w, x] ← F[w, x] + 1
            return true

return false

```

La complessità è pari al costo di tre visite, che realizzate sulla matrici hanno costo $O(n^2)$. E' possibile scrivere soluzioni che funzionano in tempo $O(m + n)$.